LeNet-5 Implementation on FPGA: An Overview

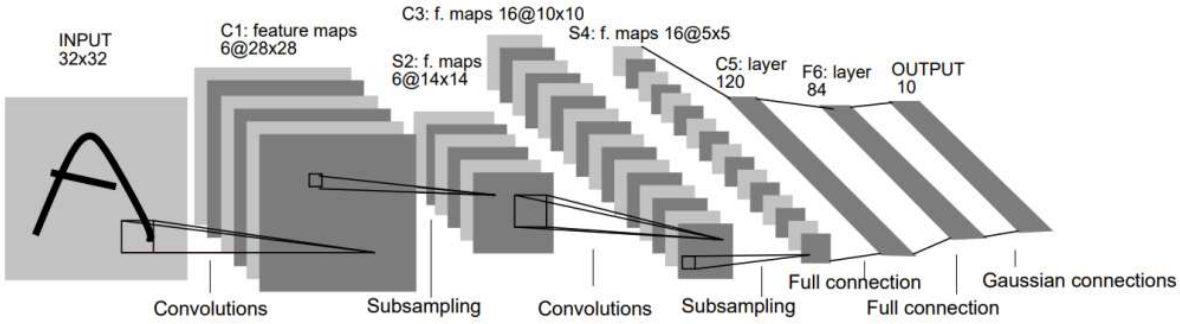
Why LeNet-5 is an ideal first CNN to implement on an FPGA, and how its layers map to FPGA resources.

November 23, 2025 · 4 min · EasyFPGA | Translations: Ko

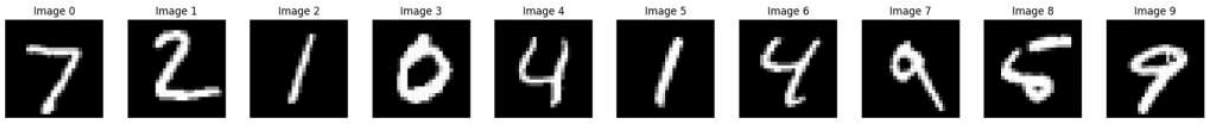
▶ Table of Contents

What Is LeNet-5?

LeNet-5 is a convolutional neural network (CNN) proposed by Yann LeCun and colleagues in 1998 for handwritten digit recognition (the MNIST dataset). It is widely regarded as the historical model that established the foundational concepts of modern deep learning: **convolution, pooling, and hierarchical feature extraction**.



LeNet-5 architecture as shown in (LeCun et al., 1998)



MNIST — 70,000 greyscale 28×28 images of handwritten digits

Network Architecture

LeNet-5 takes a **32×32 greyscale image** as input and processes it through 7 layers:

Layer	Type	Details	Output Shape
Input	—	32×32×1 greyscale	32×32×1
C1	Convolution	6 filters, 5×5 kernel, stride 1, no padding	28×28×6
S2	Average Pooling	2×2, stride 2	14×14×6
C3	Convolution	16 filters, 5×5 kernel	10×10×16
S4	Average Pooling	2×2, stride 2	5×5×16

Layer	Type	Details	Output Shape
C5	Convolution	120 filters, 5×5 kernel (fully-connected equivalent)	1×1×120
F6	Fully Connected	84 neurons	84
Output	Fully Connected	10 neurons (softmax)	10

- **Activation function:** tanh (original paper); ReLU commonly used in modern re-implementations
- **Learning algorithm:** Backpropagation
- **Total parameters:** ~61,000
- **Historical applications:** ATM cheque reading, postal code recognition

Why Implement LeNet-5 on an FPGA?

LeNet-5 is an ideal starting point for FPGA-based neural network hardware because it is **small enough to fit entirely on-chip**:

Resource	Estimate (INT8 weights)
Weight storage	~61,000 × 8 bits ≈ **62 KB**

Resource	Estimate (INT8 weights)
BRAM (18 kb blocks)	~28 blocks for weights + some for line buffers
Total BRAM budget	**~50–60 blocks** (fits in a mid-range Artix/Kintex)
MAC operations per frame	~340,000 multiply-accumulates

Because all weights fit in on-chip BRAM, the design avoids the latency and bandwidth overhead of an external DDR interface — making it much simpler to build and analyse.

Mapping CNN Layers to FPGA Logic

Convolutional Layers (C1, C3, C5)

Each convolutional layer performs a sliding-window multiply-accumulate:

$$y[r][c][k] = \sum_{i} \sum_{j} x[r+i][c+j] \cdot w_k[i][j] + b_k$$

FPGA implementation approach:

- Store the filter weights in BRAM or distributed RAM (small enough for LUT-RAM in early layers).
- Use a **line buffer** (a shift-register chain) to hold the current window of input pixels.

- Multiply each pixel in the window by the corresponding weight using DSP48 slices.
- Accumulate partial sums with an adder tree.

To maximise throughput, unroll the convolution loop over the filter dimension so that all K filters compute in parallel.

Pooling Layers (S2, S4)

LeNet-5 uses average pooling: divide the 2×2 window sum by 4 (a 2-bit right shift). This is a trivial operation in hardware — no multiplier required.

Fully Connected Layers (F6, Output)

These are standard matrix-vector multiplications. With 84 or 10 output neurons, even a sequential implementation completes in very few clock cycles.

Precision: Why Floating-Point Is Impractical on FPGA

The original C5/F6 layers use 32-bit floating-point (FP32) weights. On an FPGA:

- FP32 multipliers consume large numbers of LUTs and DSP slices.
- Latency is high relative to fixed-point alternatives.

Fixed-point (e.g., INT8 or `ap_fixed<8,2>`) is the practical choice:

- Each multiplier becomes an 8×8-bit integer multiply — fits easily in one DSP48 slice.
- Memory footprint drops from 4 bytes to 1 byte per weight.
- Accuracy loss is minimal if quantisation is done correctly (see Quantization for CNN Inference on FPGA).

Design Flow Summary

1. **Train and quantize:** Train in PyTorch/TensorFlow (FP32), export calibrated INT8 weights.
2. **Map weights to BRAM:** Use Vivado Block Memory Generator or initialise with `$readmemh`.
3. **Implement line buffers:** Use shift-register primitives (`SRL16/32` on Xilinx) to build the sliding input window.
4. **Instantiate DSP48 MAC arrays:** One DSP per filter tap, or share a single DSP via systolic scheduling.
5. **Verify against software:** Simulate the RTL against the golden PyTorch output using a common test image.
6. **Profile timing and resources:** Target 100–150 MHz on a mid-range Artix-7 or Kintex-7.

Summary

LeNet-5 is arguably the best first CNN to build in hardware:

- Small enough to fit entirely on-chip (no external DDR required)

- Architecture is well-documented and easy to understand layer by layer
- Sufficient complexity to teach all the key FPGA CNN implementation techniques: line buffers, fixed-point MAC arrays, BRAM weight storage
- Acts as a foundation for larger networks (ResNet, MobileNet) once the basic flow is mastered

FPGA LeNet CNN Deep Learning Hardware Accelerator

« PREV
Quantization for CNN Inference on FPGA

NEXT »
Ethernet II vs IEEE 802.3: Key Differences Explained

0 reactions

0 comments

Write Preview Aa

Sign in to comment

Sign in with GitHub